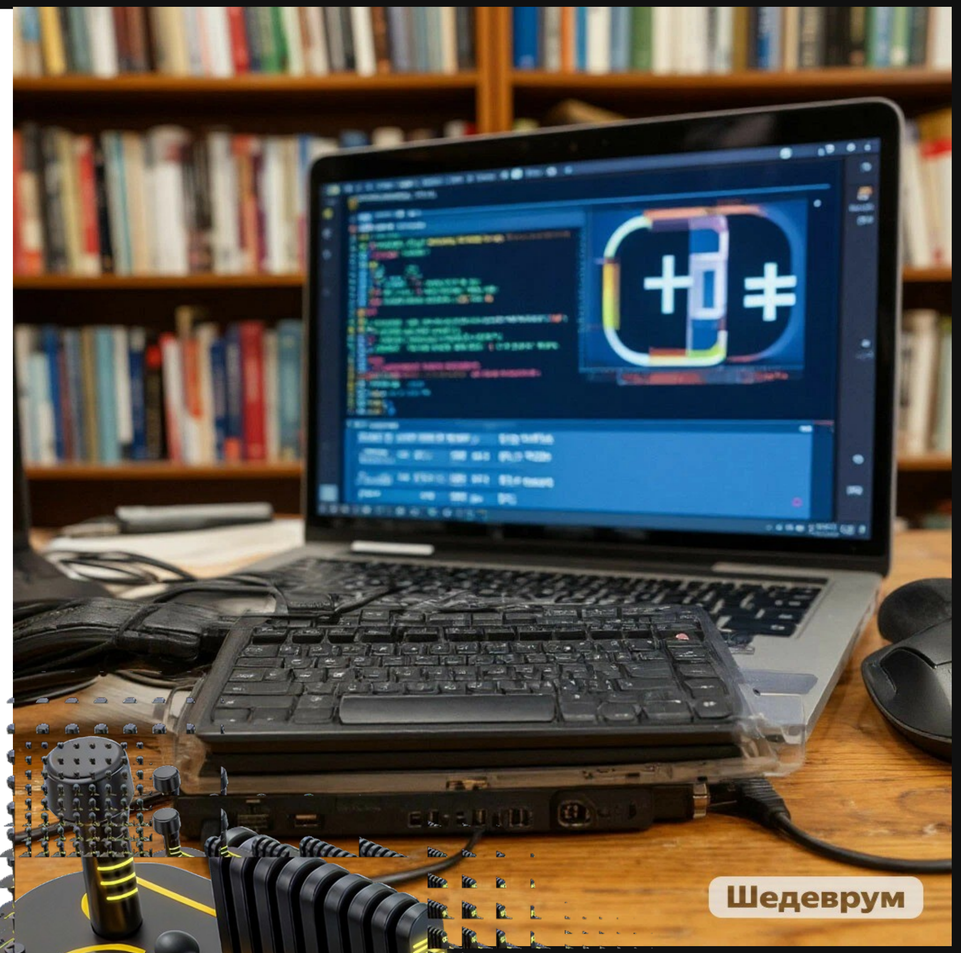
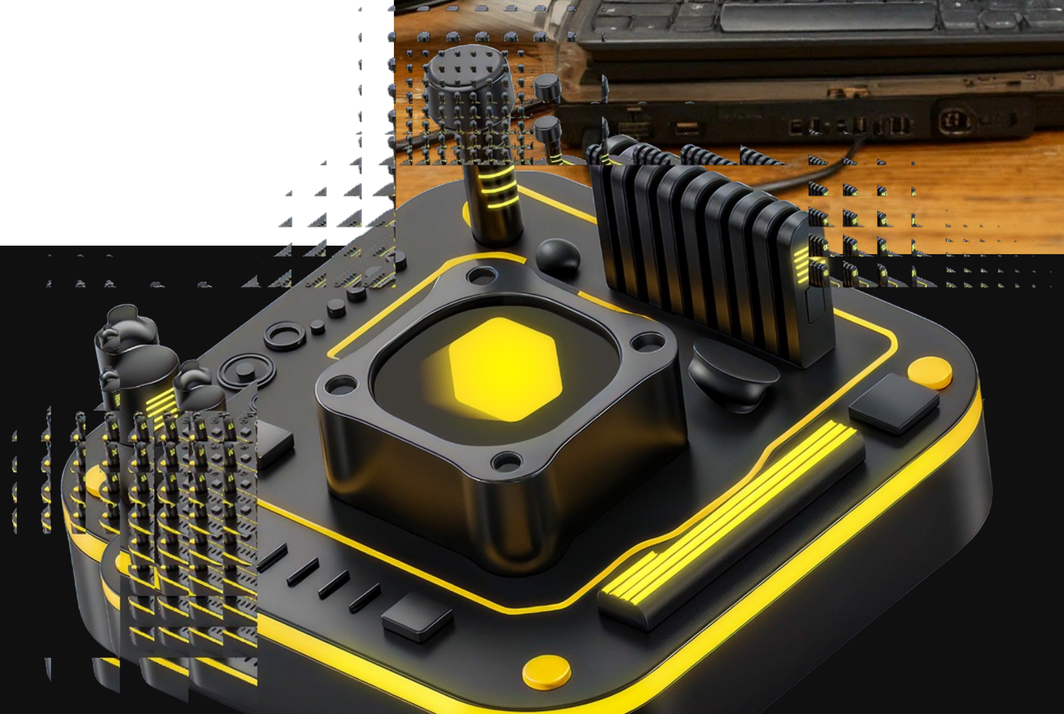
# C++ и всякое остальное

Полухин Антон
Эксперт разработчик C++



ГОРОДСКИЕ СЕРВИСЫ

# О чём поговорим


Как качать свои навыки 01

Собеседования в крупные компании 02

Смежные технологии 03

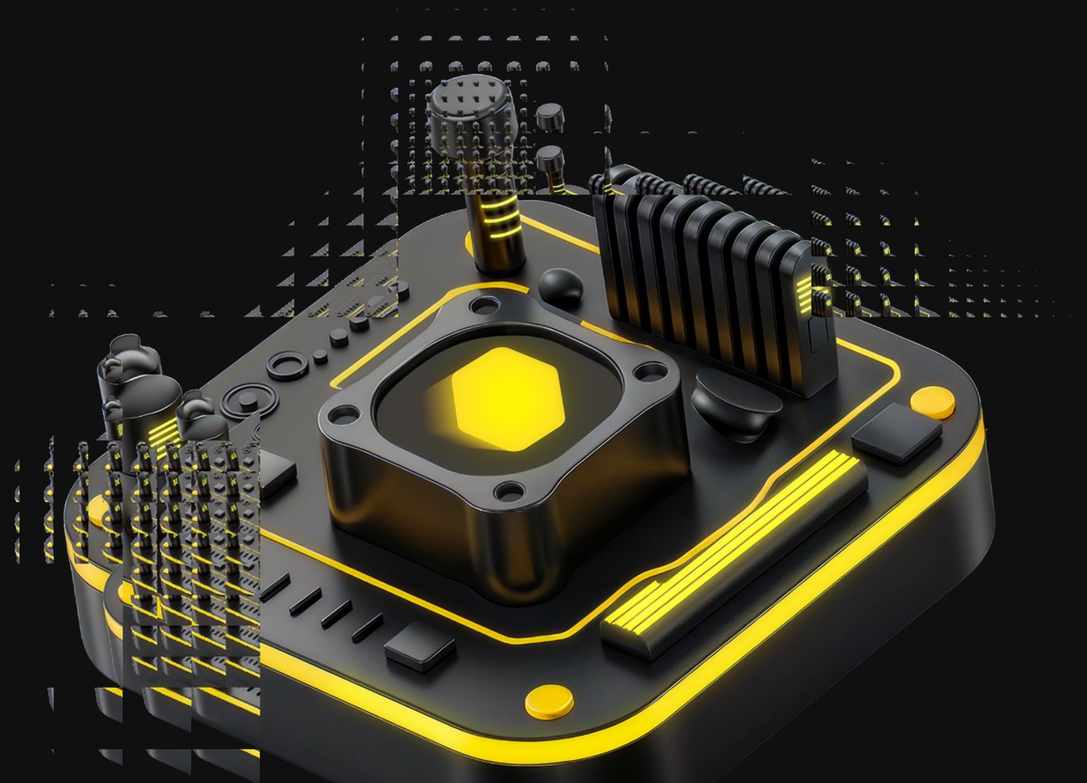
C++ 04

Многопоточность и асинхронность 05

Sanitizers + Linters 06

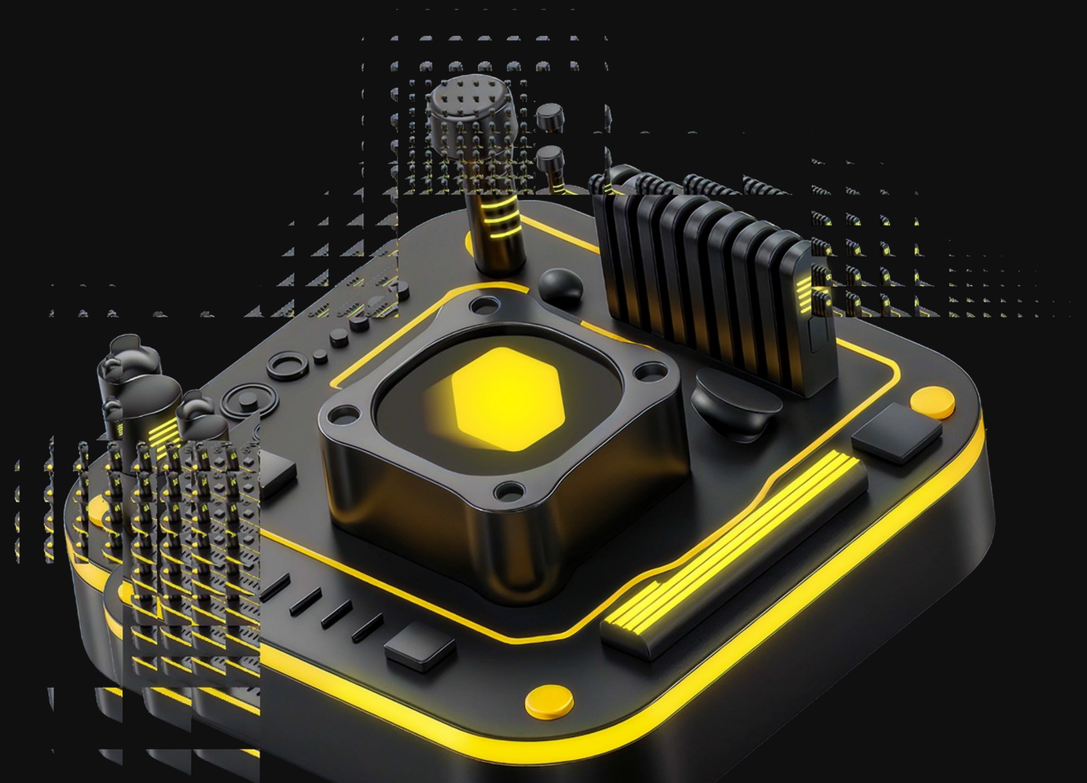
Новинки C++ 07

# Итак, вы знаете основы C++



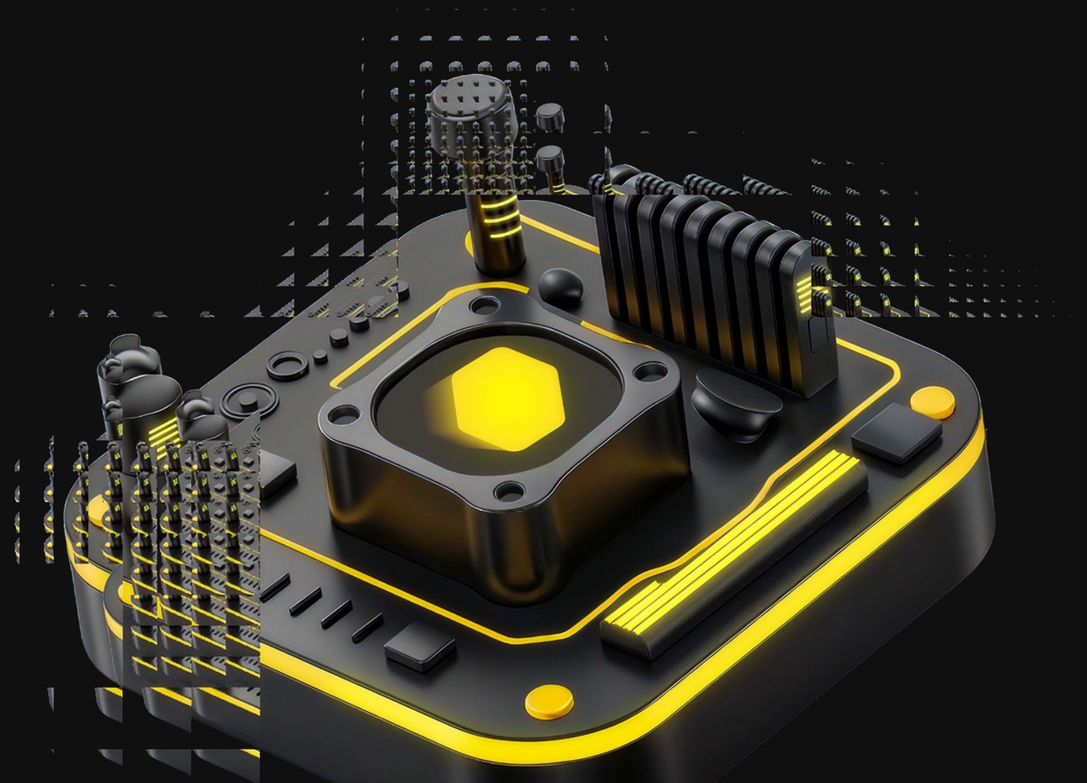
ГОРОДСКИЕ СЕРВИСЫ

# ... и хотите расти



ГОРОДСКИЕ СЕРВИСЫ

# Как прокачаться?



ГОРОДСКИЕ СЕРВИСЫ

# Что делать?

# Что делать?

01 Курсы — дорого

# Что делать?

01 Курсы — дорого

02 Книжки — нет практики

# Что делать?

01 Курсы — дорого

02 Книжки — нет практики

03 Работа — сложно найти

# Что делать?

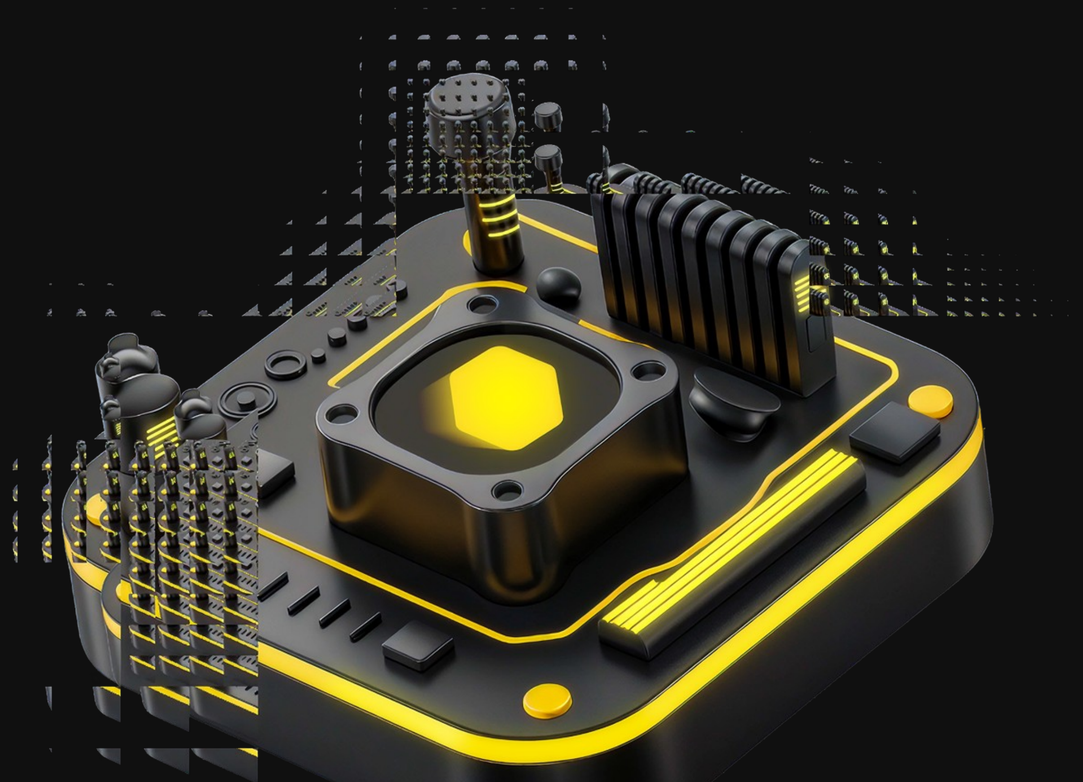
01 Курсы — дорого

02 Книжки — нет практики

03 Работа — сложно найти

04 OpenSource

# OpenSource



ГОРОДСКИЕ СЕРВИСЫ

# Берём популярный проект

# Берём популярный проект

# Берём популярный проект

# Берём популярный проект

# Берём популярный проект

# Берём популярный проект

Yandex
CatBoost

# Что получаем

# Что получаем

01 Эксперты в данной области

# Что получаем

01 Эксперты в данной области

02 Практика

# Что получаем

01 Эксперты в данной области

02 Практика

03 Практика

# Что получаем

01 Эксперты в данной области

02 Практика

03 Практика

04 Практика

# Что получаем

01 Эксперты в данной области

02 Практика

03 Практика

04 Практика

05 Бесплатно

# Что получаем

01 Эксперты в данной области

02 Практика

03 Практика

04 Практика

05 ~~Бесплатно~~ Вам не заплатят

# Что получаем

01 Эксперты в данной области

02 Практика

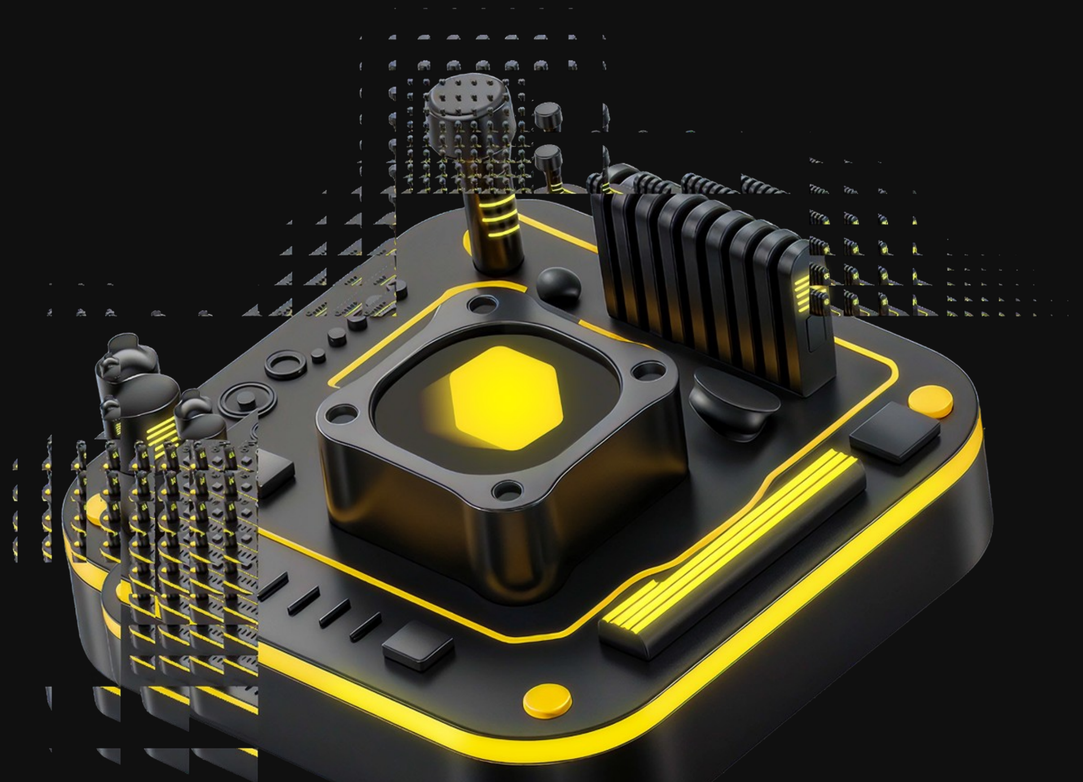
03 Практика

04 Практика

05 ~~Бесплатно~~ Вам не заплатят

06 Строчка в резюме

# Собеседуемся в компанию



ГОРОДСКИЕ СЕРВИСЫ

# В зависимости от традиций

# В зависимости от традиций

01 Предварительный скрининг

# В зависимости от традиций

01 Предварительный скрининг

02 Написание кода

# В зависимости от традиций

01 Предварительный скрининг

02 Написание кода

03 Архитектурные вопросы

# В зависимости от традиций

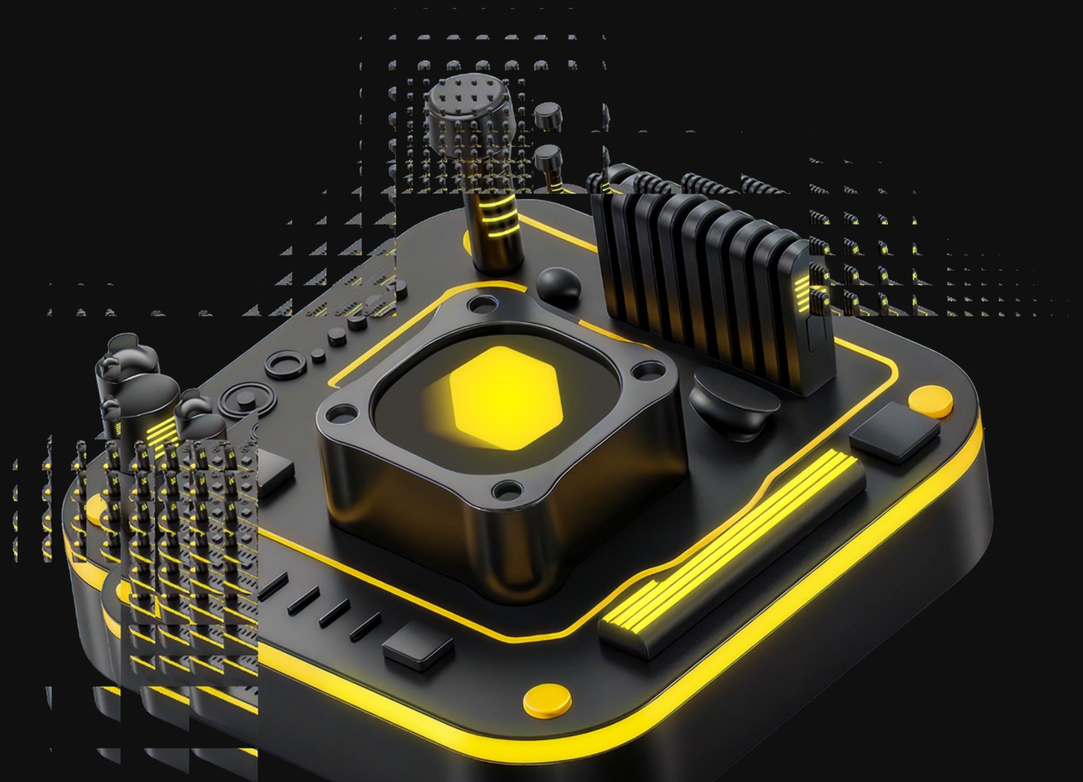
01 Предварительный скрининг

02 Написание кода

03 Архитектурные вопросы

04 Финальная секция

# Смежные технологии



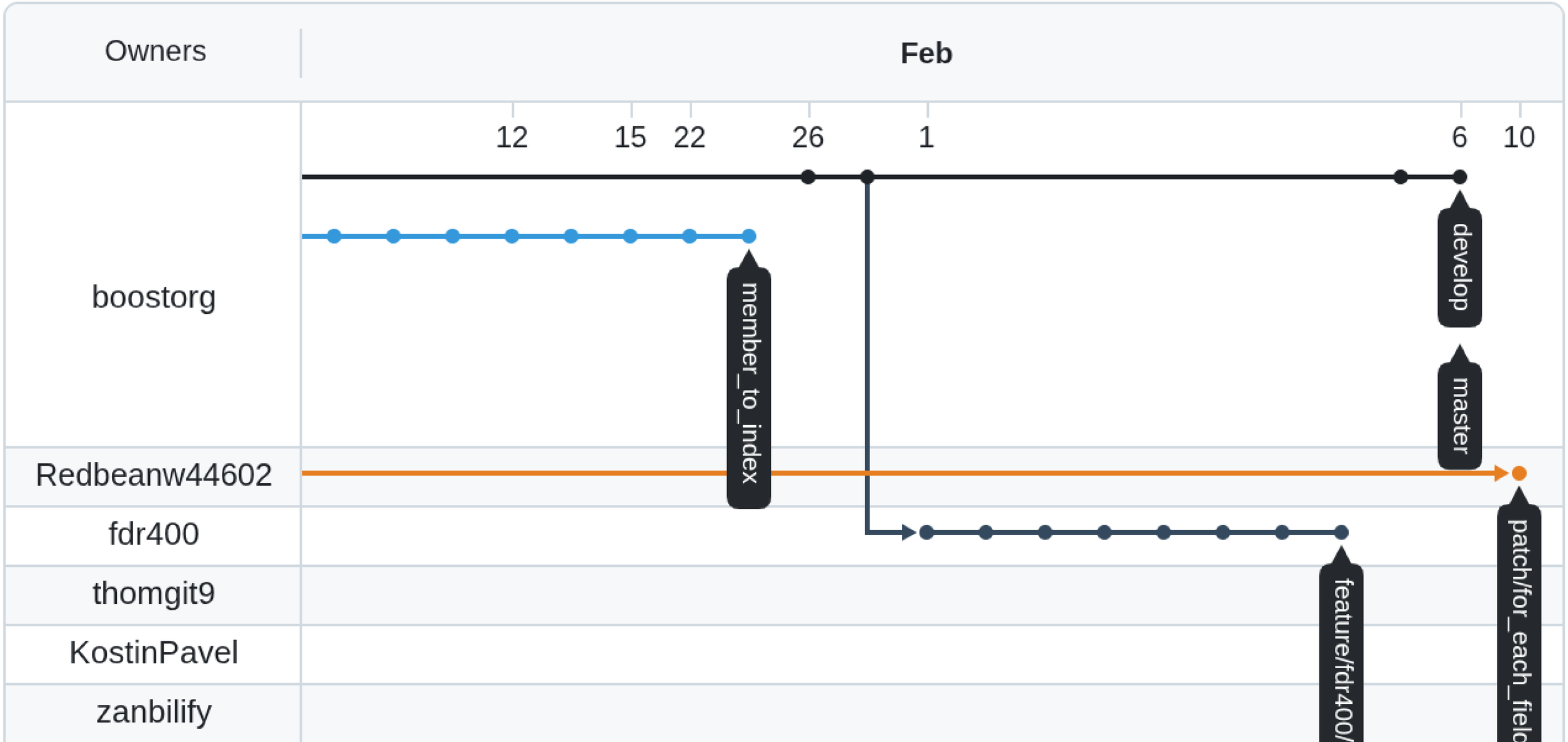
ГОРОДСКИЕ СЕРВИСЫ

# Технологии

# Технологии

01 git

# git


Owners
Feb
12
15
22
26
1
6
10
boostorg
Redbeanw44602
fdr400
thomgit9
KostinPavel
zanbilify
member_to_index
develop
master
feature/fdr400/
patch/for_each_field

# git

```
git checkout develop
git pull
git checkout -b feature-1
git add .
git commit -m “Initial implementation of the feature”
git push
git remote add me git://
git push me

git diff
git diff HEAD~1
git status
```

# git

Allow stateful visitors in for_each_field*() (#230)

develop (#230)

1 parent b21ffe6 commit adfce35

Filter files...

- include/boost/pfr
  - core.hpp
  - detail
    - core_name20_static.hpp
    - for_each_field.hpp
- test
  - core_name/run
    - for_each_field_with_nam...
  - core/run
    - for_each_field.cpp

**5 files changed** +27 -6 lines changed

Search within code

include/boost/pfr/core.hpp +1 -1

```diff
@@ -255,7 +255,7 @@ constexpr auto structure_tie(T&&, std::enable_if_t< std::is_rvalue_reference<T&&
 /// \endcode
 template <class T, class F>
 constexpr void for_each_field(T&& value, F&& func) {
-    return ::boost::pfr::detail::for_each_field(std::forward<T>(value), std::forward<F>(func));
+    ::boost::pfr::detail::for_each_field(std::forward<T>(value), std::forward<F>(func));
 }

 /// \brief std::tie-like function that allows assigning to tied values from aggregates.
```

include/boost/pfr/detail/core_name20_static.hpp +3 -3

```diff
@@ -245,14 +245,14 @@ template <class T, class F>
 constexpr void for_each_field_with_name(T&& value, F&& func) {
     return boost::pfr::detail::for_each_field(
         std::forward<T>(value),
-        [f = std::forward<F>(func)](auto&& field, auto index) mutable {
+        [&func](auto&& field, auto index) mutable {
```

# Технологии

01 git

# Технологии

01 git

02 python/bash/sed/...

# python/bash/sed

```
find . -type f | xargs sed -i ‘s/foo/bar/g’
find ./services/*/configs -type f | xargs sed -i ‘s/foo/bar/g’
find ./services/*/configs -type f | xargs sed -i ‘/password/d’
```

# Технологии

01 git

02 python/bash/sed/...

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

# SQL & NoSQL

# SQL & NoSQL

```
SELECT 1
```

# SQL & NoSQL

```
SELECT 1
BEGIN/END
```

# SQL & NoSQL

SELECT 1

BEGIN/END

SparQL, GraphQL

YQL

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

# Технологии

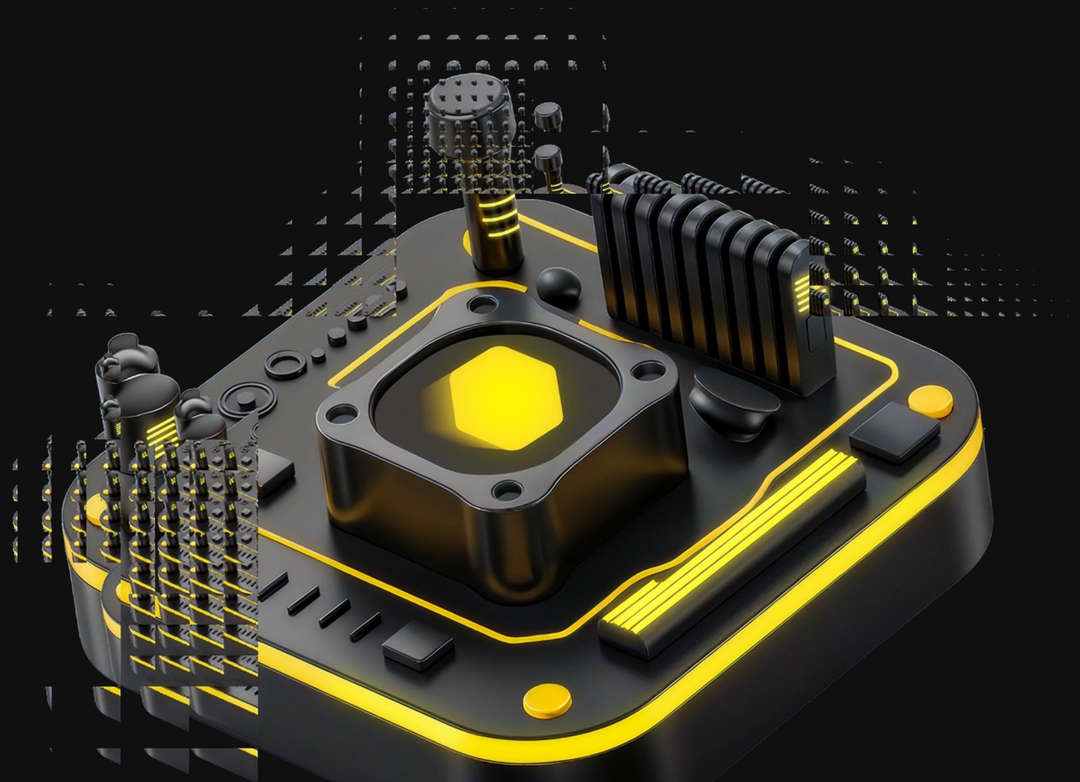
01 git

02 python/bash/sed/...

03 SQL & NoSQL

04 JSON/YAML/Protobuf/Flatbuf/...

# C++ - это...



ГОРОДСКИЕ СЕРВИСЫ

# C++ основные области:

# C++ основные области:

01 Performance critical

# C++ основные области:

01 Performance critical

02 Low Latency / Jitter

# C++ основные области:

01 Performance critical

02 Low Latency / Jitter

03 Memory consuming

# C++ основные области:

01 Performance critical

02 Low Latency / Jitter

03 Memory consuming

04 Hardware / Compatibility

# C++ основные области:

01 Performance critical

02 Low Latency / Jitter

03 Memory consuming

04 Hardware / Compatibility

05 Power effective

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

04 JSON/YAML/Protobuf/Flatbuf/...

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

04 JSON/YAML/Protobuf/Flatbuf/...

05 OS

# OS

# OS

01 syscall

# OS

01 syscall

02 Context switch

# OS

01 syscall

02 Context switch

03 vDSO

# OS

01 syscall

02 Context switch

03 vDSO

04 Major Page Fault

# OS

01 syscall

02 Context switch

03 vDSO

04 Major Page Fault

05 select/poll/epoll/...

# OS

01 syscall

02 Context switch

03 vDSO

04 Major Page Fault

05 select/poll/epoll/...

06 ???

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

04 JSON/YAML/Protobuf/Flatbuf/...

05 OS

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

04 JSON/YAML/Protobuf/Flatbuf/...

05 OS

06 Assembler

godbolt.org — Compiler Explorer

COMPILER EXPLORER — Add... More Templates — Share Policies Other

C++ source #1

```cpp
struct NoConstexpr {
    NoConstexpr(){}
};
struct WithConstexpr {
    constexpr WithConstexpr(){}
};

auto& test_no() {
    static NoConstexpr g;
    return g;
}

auto& test_with() {
    static WithConstexpr g;
    return g;
}
```

x86-64 gcc (trunk) (Editor #1) — x86-64 gcc (trunk) — -std=c++20 -O2

```asm
"test_no()":
        movzx   eax, BYTE PTR "guard varia
        test    al, al
        je      .L11
        mov     eax, OFFSET FLAT:"test_no(
        ret
.L11:
        sub     rsp, 8
        mov     edi, OFFSET FLAT:"guard va
        call    "__cxa_guard_acquire"
        test    eax, eax
        jne     .L12
        mov     eax, OFFSET FLAT:"test_no(
        add     rsp, 8
        ret
.L12:
        mov     edi, OFFSET FLAT:"guard va
        call    "__cxa_guard_release"
        mov     eax, OFFSET FLAT:"test_no(
        add     rsp, 8
        ret
"test_with()":
        mov     eax, OFFSET FLAT:"test_wit
        ret
```

x86-64 clang 19.1.0 (Editor #1) — x86-64 clang 19.1.0 — -O2 -std=c+

```asm
test_no():
        movzx   eax, byte ptr [rip + g
        test    al, al
        je      .LBB0_1
        lea     rax, [rip + test_no():
        ret
.LBB0_1:
        push    rax
        lea     rdi, [rip + guard vari
        call    __cxa_guard_acquire@PL
        test    eax, eax
        je      .LBB0_3
        lea     rdi, [rip + guard vari
        call    __cxa_guard_release@PL
.LBB0_3:
        add     rsp, 8
        lea     rax, [rip + test_no():
        ret

test_with():
        lea     rax, [rip + test_with(
        ret
```

Output (0/0) x86-64 clang 19.1.0 — cached (5202B) ~323 lines

filtered — Compiler License

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

04 JSON/YAML/Protobuf/Flatbuf/...

05 OS

06 Assembler

# Технологии

01 git

02 python/bash/sed/...

03 SQL & NoSQL

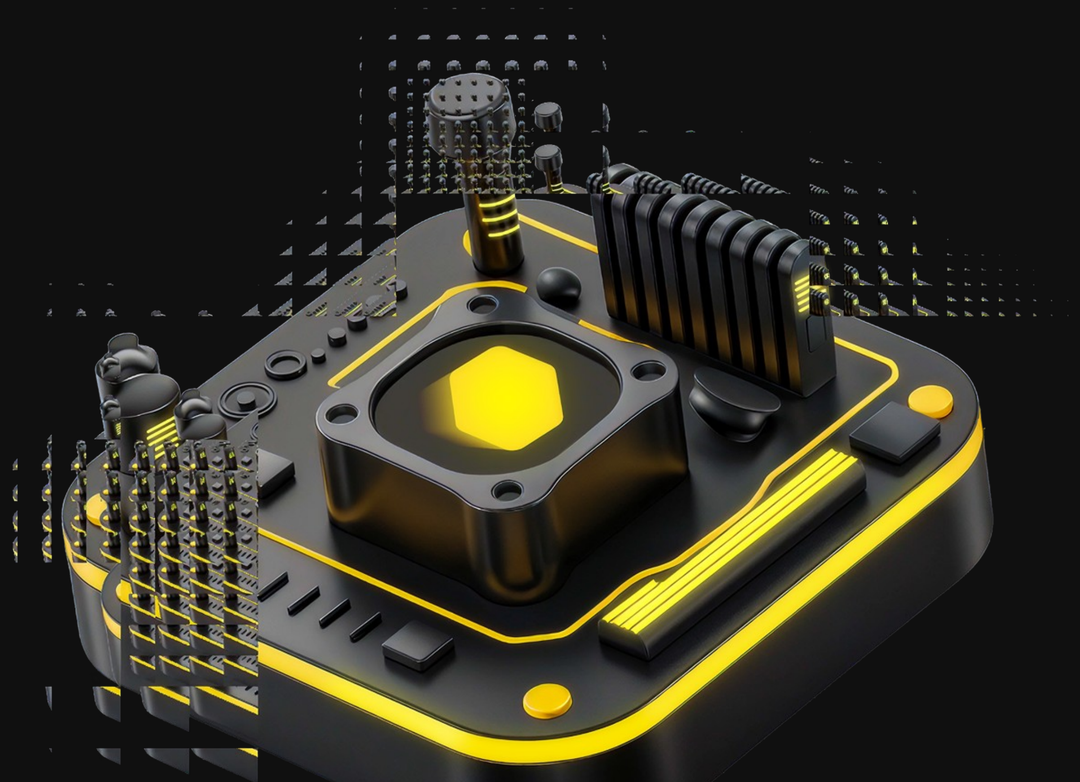
04 JSON/YAML/Protobuf/Flatbuf/...

05 OS

06 Assembler

07 ???

# C++



# C++

# C++

```
char* get1();  char* get2();  void do_something(const char* s);
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}

void example1() {
    char *s1 = get1(), *s2 = get2();
    char* result = str_plus(s1, s2);
    do_something(result);
    free(result);
    // free(s1); ???
    // free(s2); ???
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}

void example1() {
    char *s1 = get1(), *s2 = get2();
    char* result = str_plus(s1, s2);
    do_something(result);
    free(result);
    // free(s1); ???
    // free(s2); ???
}
```

# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}

void example1() {
    char *s1 = get1(), *s2 = get2();
    char* result = str_plus(s1, s2);
    do_something(result);
    free(result);
    // free(s1); ???
    // free(s2); ???
}
```

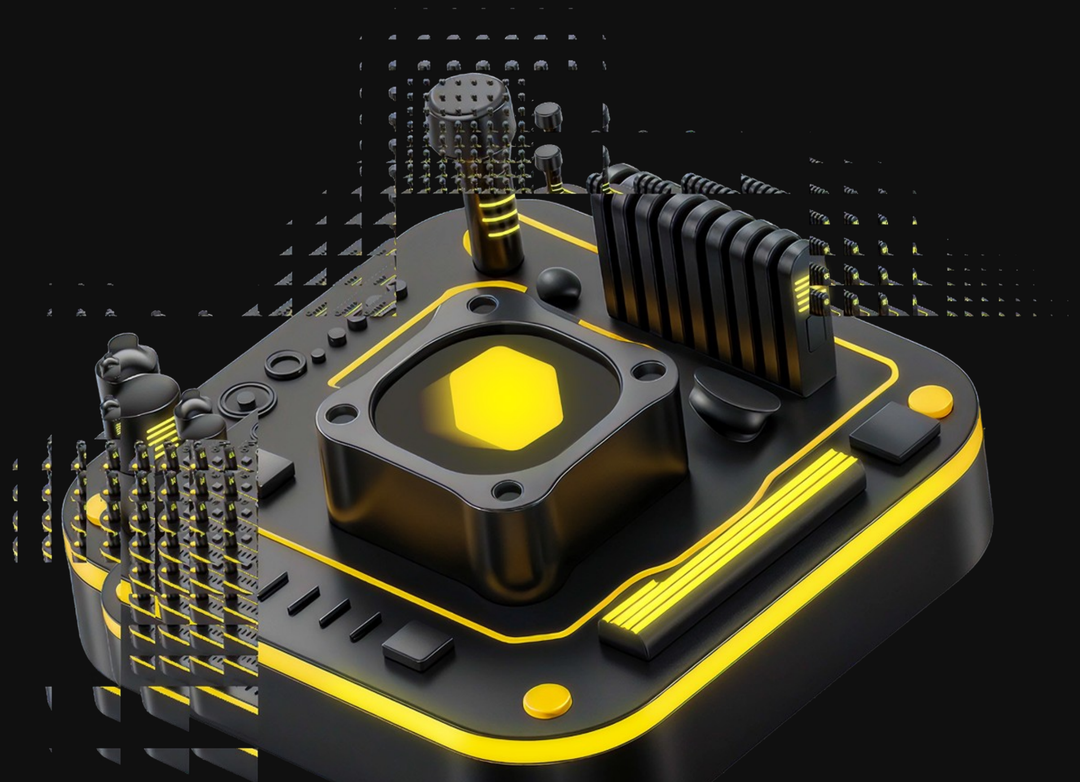
# C++

```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}

void example1() {
    char *s1 = get1(), *s2 = get2();
    char* result = str_plus(s1, s2);
    do_something(result);
    free(result);
    // free(s1); ???
    // free(s2); ???
}
```

# C++

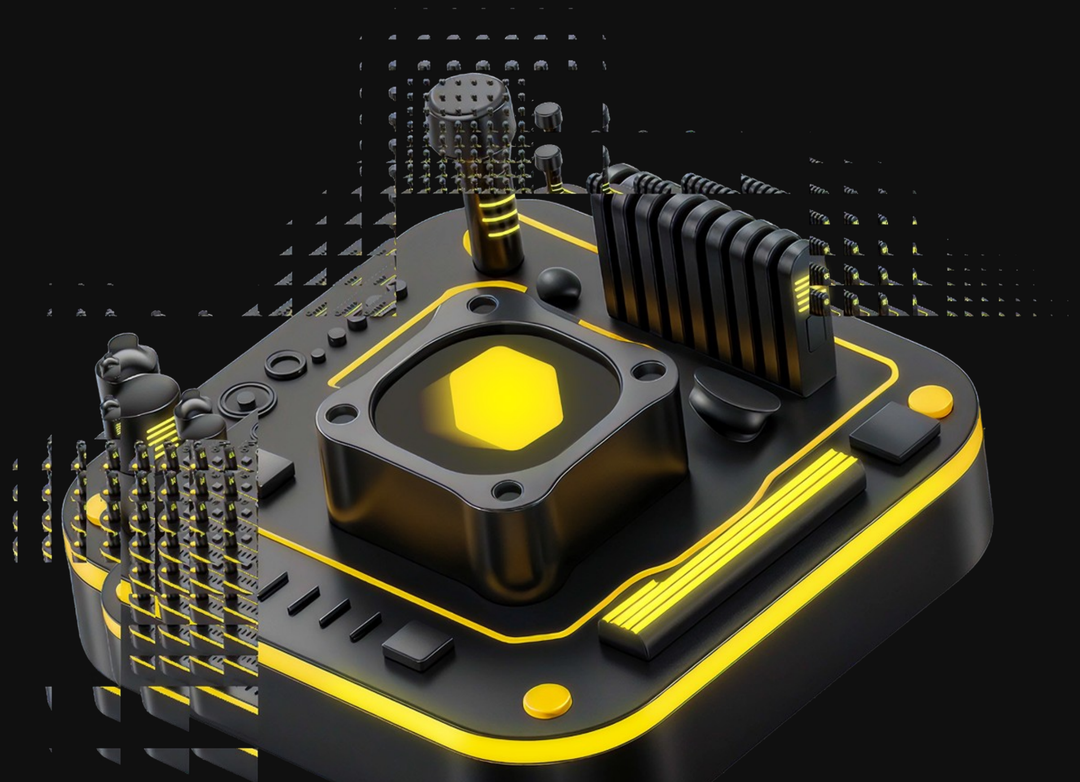
```
char* get1();  char* get2();  void do_something(const char* s);

char* str_plus(const char* s1, const char* s2) {
    unsigned len = strlen(s1) + strlen(s2) + 1;
    char* result = (char*)malloc(len);
    strcat(result, s1);
    strcat(result, s2);
    return result;
}

void example1() {
    char *s1 = get1(), *s2 = get2();
    char* result = str_plus(s1, s2);
    do_something(result);
    free(result);
    // free(s1); ???
    // free(s2); ???
}
```

# Это был не C++



ГОРОДСКИЕ СЕРВИСЫ

# Вот C++



# C++

```
std::string get_str1();  std::string get_str2();  void do_something(const char* s);

void example2() {
    auto result = get_str1() + get_str2();
    do_something(result.c_str());
}
```

# C++

```
std::string get_str1();  std::string get_str2();  void do_something(const char* s);

void example2() {
    auto result = get_str1() + get_str2();
    do_something(result.c_str());
}
```

# C++

```
std::string get_str1();  std::string get_str2();  void do_something(const char* s);

void example2() {
    auto result = get_str1() + get_str2();
    do_something(result.c_str());
}
```

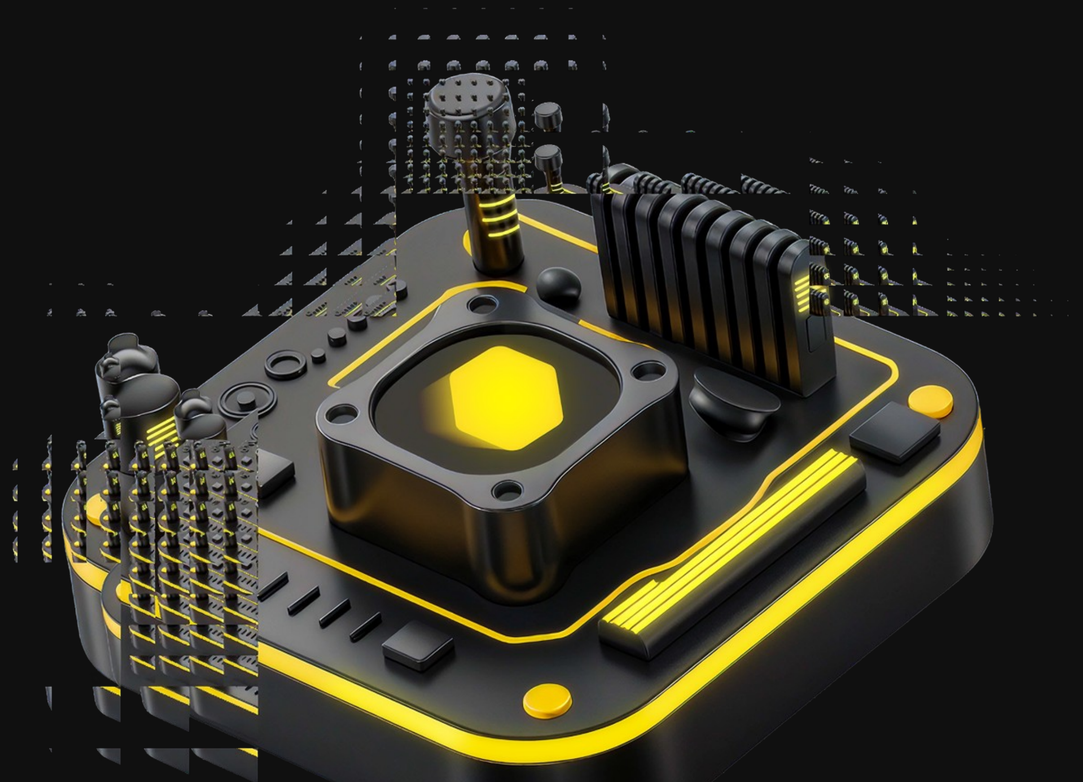
# C++

```
std::string get_str1();  std::string get_str2();  void do_something(const char* s);

void example2() {
    auto result = get_str1() + get_str2();
    do_something(result.c_str());
}
```

# C++

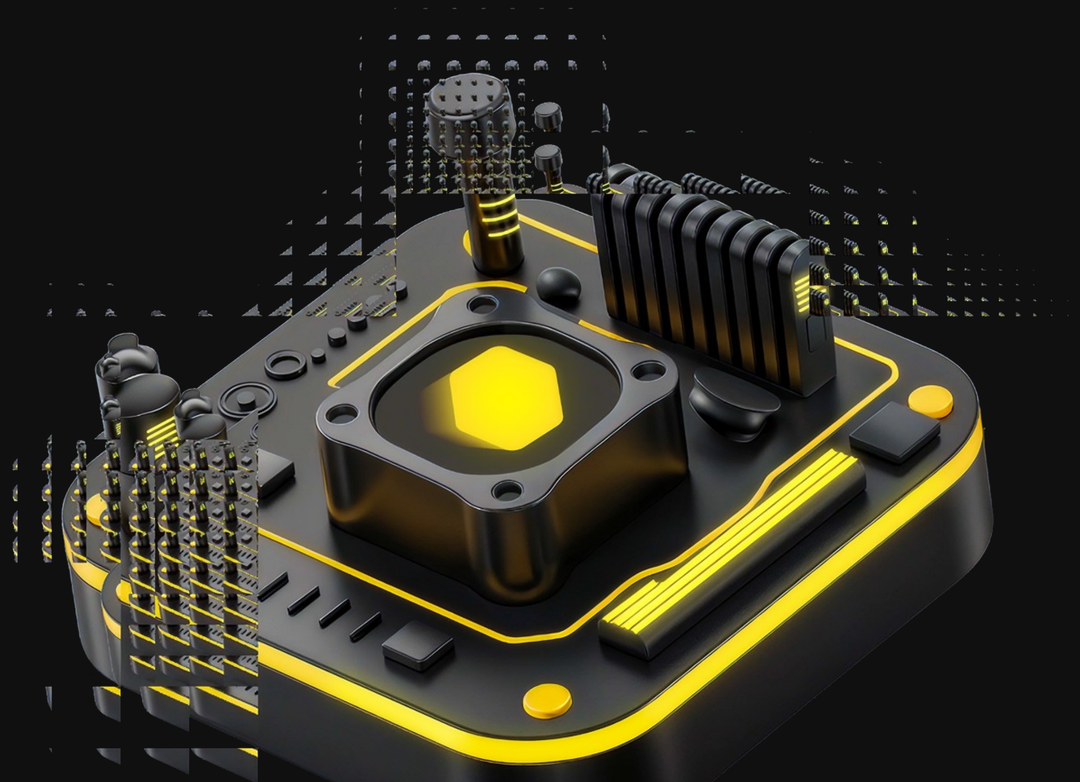
```
std::string get_str1();  std::string get_str2();  void do_something(const char* s);

void example2() {
    auto result = get_str1() + get_str2();
    do_something(result.c_str());
}
```

# Самый главный принцип C++ ?



ГОРОДСКИЕ СЕРВИСЫ

# RAII



# Найди ошибку

```
01: char* str_plus(const char* s1, const char* s2) {
02:     unsigned len = strlen(s1) + strlen(s2) + 1;
03:     char* result = (char*)malloc(len);
04:     strcat(result, s1);
05:     strcat(result, s2);
06:     return result;
07: }
08:
09: void example1() {
10:     char *s1 = get1(), *s2 = get2();
11:     char* result = str_plus(s1, s2);
12:     do_something(result);
13:     free(result);
14:     // free(s1); ???
15:     // free(s2); ???
16: }
```

# Найди ошибку

```
01: char* str_plus(const char* s1, const char* s2) {
02:     unsigned len = strlen(s1) + strlen(s2) + 1;
03:     char* result = (char*)malloc(len);
04:     strcat(result, s1);
05:     strcat(result, s2);
06:     return result;
07: }
08:
09: void example1() {
10:     char *s1 = get1(), *s2 = get2();
11:     char* result = str_plus(s1, s2);
12:     do_something(result);
13:     free(result);
14:     // free(s1); ???
15:     // free(s2); ???
16: }
```

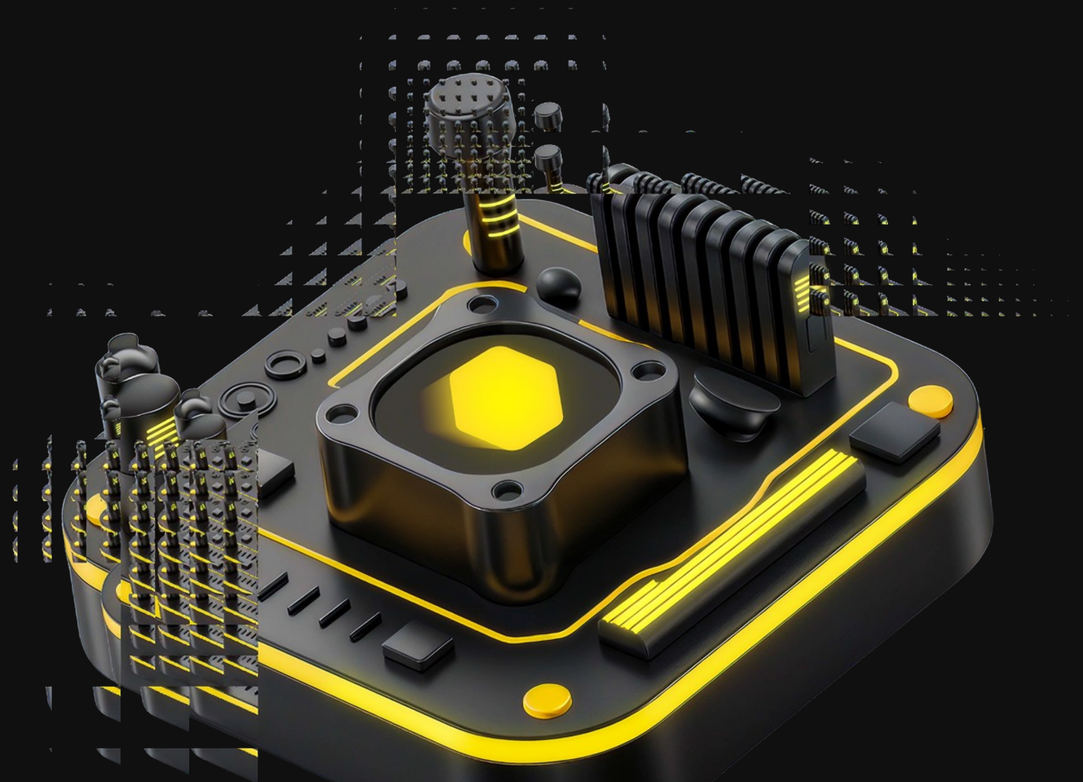
# Найди ошибку

```
01: char* str_plus(const char* s1, const char* s2) {
02:     unsigned len = strlen(s1) + strlen(s2) + 1;
03:     char* result = (char*)malloc(len);
04:     strcat(result, s1);
05:     strcat(result, s2);
06:     return result;
07: }
08:
09: void example1() {
10:     char *s1 = get1(), *s2 = get2();
11:     char* result = str_plus(s1, s2);
12:     do_something(result);
13:     free(result);
14:     // free(s1); ???
15:     // free(s2); ???
16: }
```

# C++

```
std::string get_str1();  std::string get_str2();  void do_something(const char* s);

void example2() {
    auto result = get_str1() + get_str2();
    do_something(result.c_str());
}
```

# C++ контейнеры



ГОРОДСКИЕ СЕРВИСЫ

# C++ smart ptr

# C++ smart ptr

```
#include <memory>


class MyClass {
    // ...
private:
    std::shared_ptr<Some2> shared_ = std::make_shared<Some2>();




};
```

# C++ smart ptr

```
#include <memory>


class MyClass {
    // ...
private:
    std::shared_ptr<Some2> shared_ = std::make_shared<Some2>();
    std::shared_ptr<const Some3> const_shared_ = std::make_shared<Some3>();



};
```

# C++ smart ptr

```
#include <memory>


class MyClass {
    // ...
private:
    std::shared_ptr<Some2> shared_ = std::make_shared<Some2>();
    std::shared_ptr<const Some3> const_shared_ = std::make_shared<Some3>();

    std::unique_ptr<Some0> impl_ = std::make_unique<Some0>();


};
```

# C++ smart ptr

```
#include <memory>


class MyClass {
    // ...
private:
    std::shared_ptr<Some2> shared_ = std::make_shared<Some2>();
    std::shared_ptr<const Some3> const_shared_ = std::make_shared<Some3>();

    std::unique_ptr<Some0> impl_ = std::make_unique<Some0>();

    std::unique_ptr<const Some1> some_impl_ = std::make_unique<Some1>();
};
```

# C++ smart ptr

```
#include <memory>


class MyClass {
    // ...
private:
    std::shared_ptr<Some2> shared_ = std::make_shared<Some2>();
    std::shared_ptr<const Some3> const_shared_ = std::make_shared<Some3>();

    std::unique_ptr<Some0> impl_ = std::make_unique<Some0>();

    std::unique_ptr<const Some1> some_impl_ = std::make_unique<Some1>();
};
```

# C++ самый популярный контейнер

# C++ самый популярный контейнер

```
std::vector<std::size_t> the_most_popular_container;
```

# C++ самый популярный контейнер

```
std::vector<std::size_t> the_most_popular_container;

for (std::size_t i = 0; i < 1024; ++i) {
    the_most_popular_container.push_back(i);
}
```

# C++ самый популярный контейнер

```
std::vector<std::size_t> the_most_popular_container;

the_most_popular_container.reserve(1024);
for (std::size_t i = 0; i < 1024; ++i) {
    the_most_popular_container.push_back(i);
}
```

# C++ самый популярный контейнер

```
std::vector<std::size_t> the_most_popular_container;

the_most_popular_container.reserve(1024);
for (std::size_t i = 0; i < 1024; ++i) {
    the_most_popular_container.push_back(i);
}
```

# C++ самый популярный контейнер

```
std::vector<std::size_t> the_most_popular_container;

the_most_popular_container.reserve(1024);
for (std::size_t i = 0; i < 1024; ++i) {
    the_most_popular_container.push_back(i);
}
```

# C++ самый собес. контейнер

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

# C++ самый собес. контейнер

```
struct string_hash {
  using is_transparent = std::true_type;


  std::size_t operator()(std::string_view txt) const noexcept {
      return std::hash<std::string_view>{}(txt);
  }
};
```

# C++ самый собес. контейнер

```
struct string_hash {
  using is_transparent = std::true_type;


  std::size_t operator()(std::string_view txt) const noexcept {
      return std::hash<std::string_view>{}(txt);
  }
};
```

# C++ самый собес. контейнер

```
struct string_hash {
  using is_transparent = std::true_type;


  std::size_t operator()(std::string_view txt) const noexcept {
      return std::hash<std::string_view>{}(txt);
  }
};
```

# C++ самый собес. контейнер

```
struct string_hash {
  using is_transparent = std::true_type;


  std::size_t operator()(std::string_view txt) const noexcept {
      return std::hash<std::string_view>{}(txt);
  }
};
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

```
std::unordered_map<std::string, int, string_hash, std::equal_to<>> kv2;
kv2["some_long_long_long_long_long_key"] = 1;
kv2.find("some_long_long_long_long_long_long_key") != kv2.end();
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

```
std::unordered_map<std::string, int, string_hash, std::equal_to<>> kv2;
kv2["some_long_long_long_long_long_key"] = 1;
kv2.find("some_long_long_long_long_long_long_key") != kv2.end();
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

```
std::unordered_map<std::string, int, string_hash, std::equal_to<>> kv2;
kv2["some_long_long_long_long_long_key"] = 1;
kv2.find("some_long_long_long_long_long_long_key") != kv2.end();
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

```
std::unordered_map<std::string, int, string_hash, std::equal_to<>> kv2;
kv2["some_long_long_long_long_long_key"] = 1;
kv2.find("some_long_long_long_long_long_long_key") != kv2.end();
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

```
std::unordered_map<std::string, int, string_hash, std::equal_to<>> kv2;
kv2["some_long_long_long_long_long_key"] = 1;
kv2.find("some_long_long_long_long_long_long_key") != kv2.end();
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

```
std::unordered_map<std::string, int, string_hash, std::equal_to<>> kv2;
kv2["some_long_long_long_long_long_key"] = 1;
kv2.find("some_long_long_long_long_long_long_key") != kv2.end();
```

# C++ самый собес. контейнер

```
std::unordered_map<std::string, int> kv;
kv["some_long_long_long_long_long_key"] = 1;
kv.find("some_long_long_long_long_long_long_key") != kv.end();
```

```
std::unordered_map<std::string, int, string_hash, std::equal_to<>> kv2;
kv2["some_long_long_long_long_long_key"] = 1;
kv2.find("some_long_long_long_long_long_long_key") != kv2.end();
```

# std::string vs std::string_view

```
const std::string kKeyLongLong = "some_long_long_long_long_long_long_key";
```

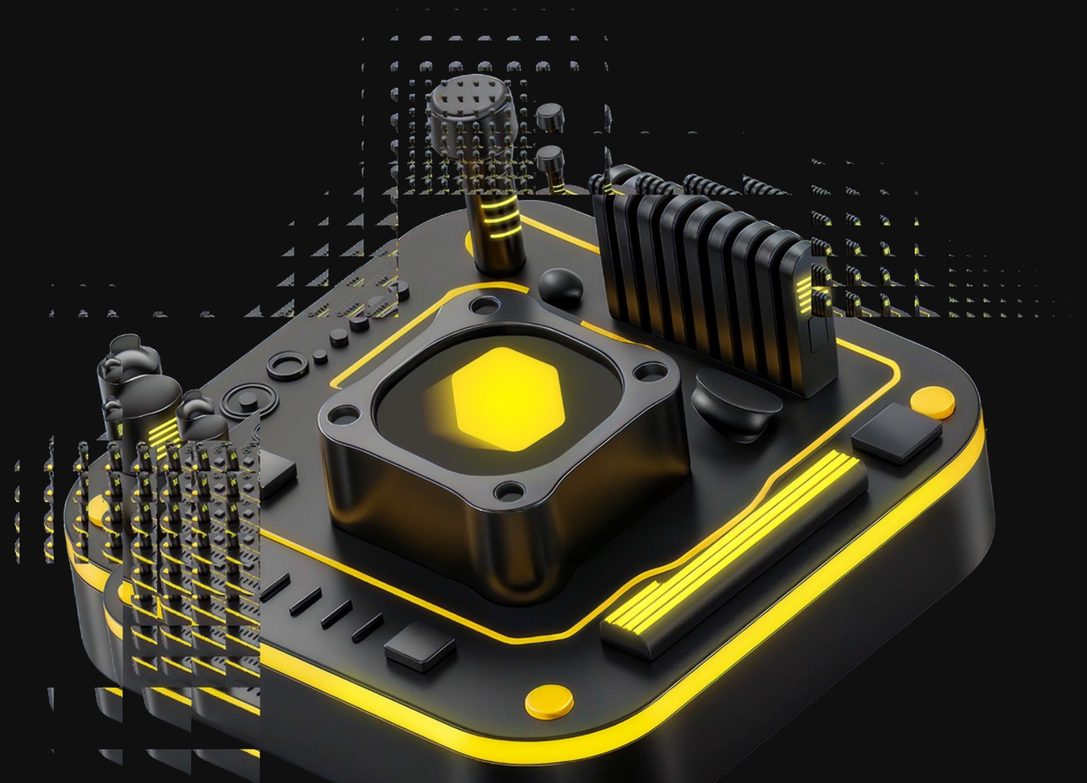
# std::string vs std::string_view

```
constexpr std::string_view kKeyLongLong = "some_long_long_long_long_long_long_key";
```

# std::string vs std::string_view

```
constexpr std::zstring_view kKeyLongLong = "some_long_long_long_long_long_long_key";
```

# Многопоточность и асинхронность



ГОРОДСКИЕ СЕРВИСЫ

# std::atomic

# std::atomic

```
std::atomic<int> atomic{42};
```

# std::atomic

```
std::atomic<int> atomic{42};
static_assert(std::atomic<int>::is_always_lock_free);
```

# std::atomic

```
std::atomic<int> atomic{42};
static_assert(std::atomic<int>::is_always_lock_free);
```

```
std::memory_order_seq_cst;
```

# std::atomic

```
std::atomic<int> atomic{42};
static_assert(std::atomic<int>::is_always_lock_free);
```

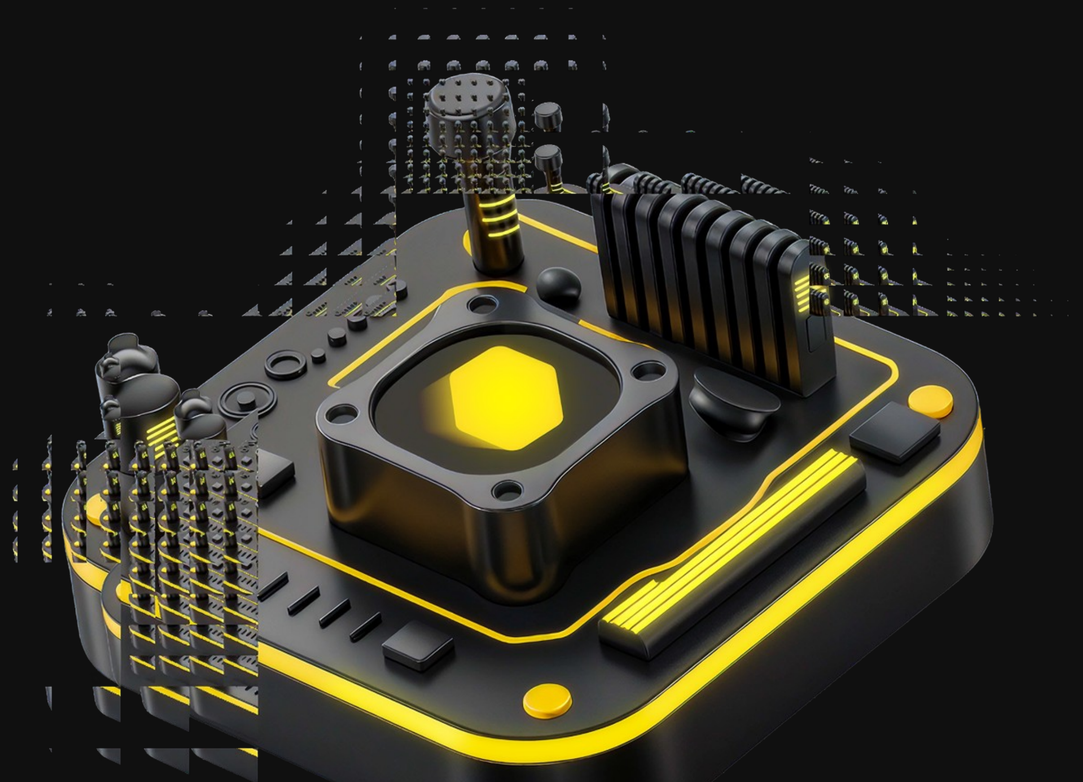
# std::atomic

```
std::atomic<int> atomic{42};
static_assert(std::atomic<int>::is_always_lock_free);

atomic.store(0)
atomic.store(0, std::memory_order_relaxed);
```

# std::atomic

```
std::atomic<int> atomic{42};
static_assert(std::atomic<int>::is_always_lock_free);

atomic.store(0)
atomic.store(0, std::memory_order_relaxed);

volatile int not_atomic;
```

# Sanitizers & Linters



# Sanitizers

01 ASAN

02 UBSAN

03 TSAN

04 MSAN ?

# Linters

01 concurrency-mt-unsafe

02 bugprone-move-forwarding-reference

03 bugprone-use-after-move

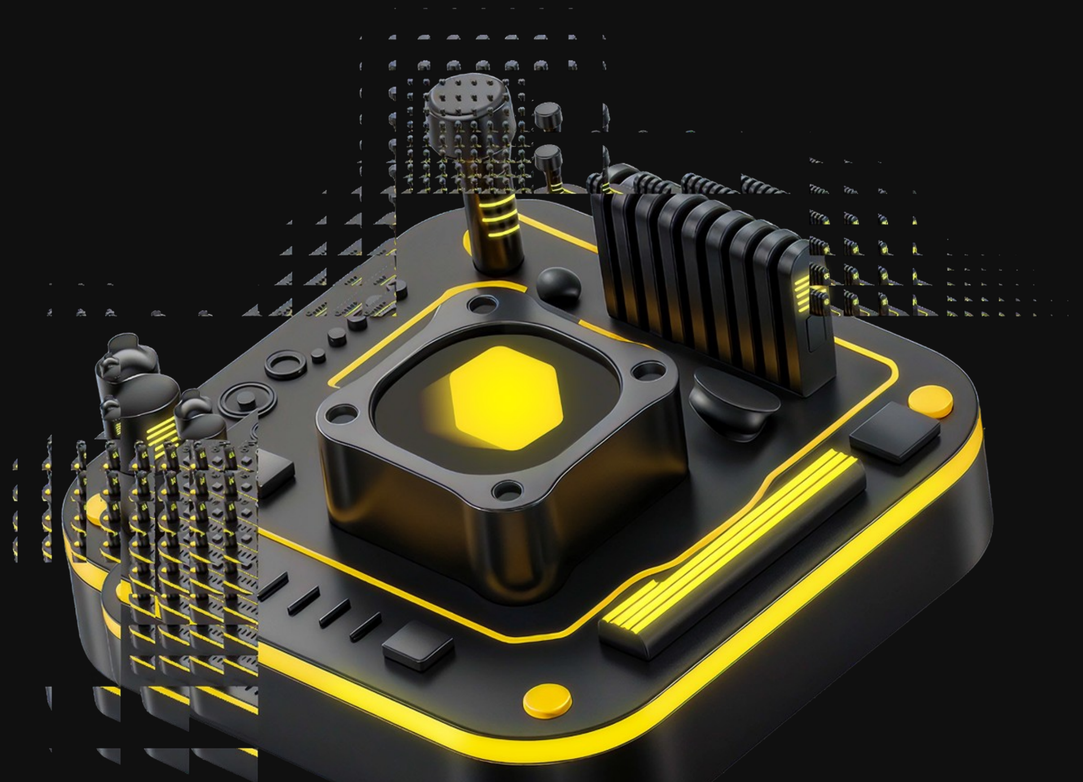
04 bugprone-unchecked-optional-access

05 performance-implicit-conversion-in-loop

06 performance-noexcept-move-constructor

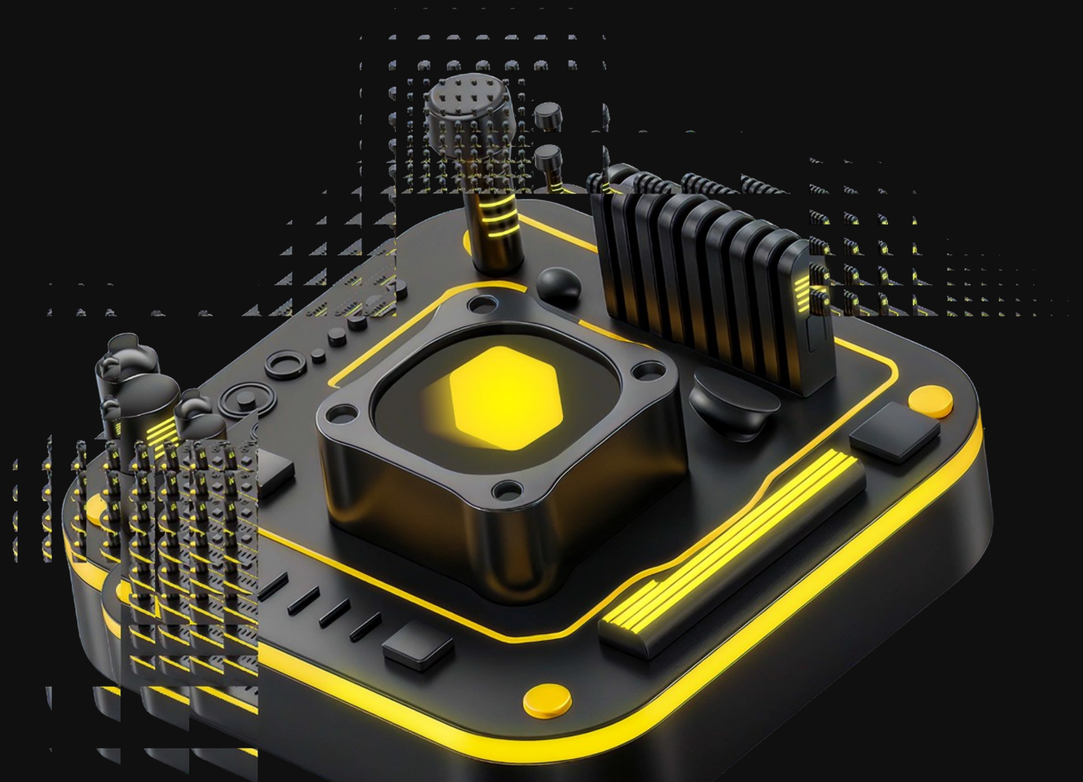
07 bugprone-string-integer-assignment

# Новинки C++



ГОРОДСКИЕ СЕРВИСЫ

# Рефлексия



# Reflection

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return




      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return

            ^^T,



      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return

            ^^T,



      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return

            ^^T,



      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        )
      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        )
      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        )
      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        ).at(I)
      ;
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return     [:
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        ).at(I)
    :];
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return     [:
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        ).at(I)
    :];
}
```

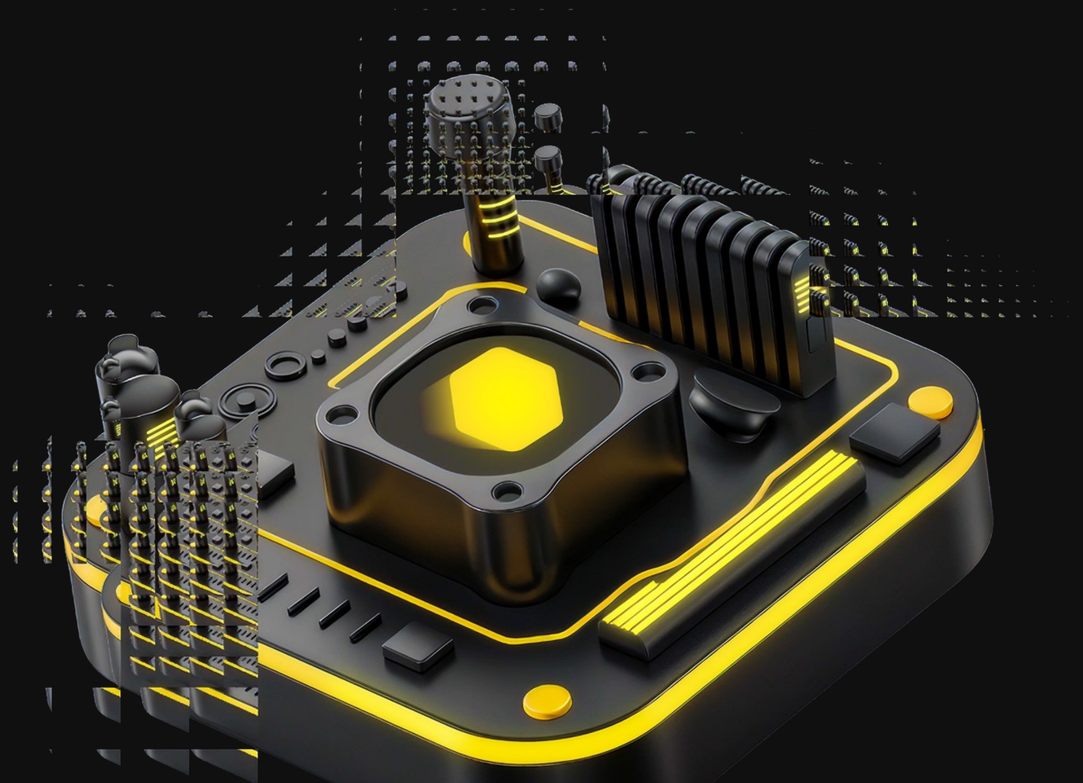
# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return     [:
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        ).at(I)
    :];
}
```

# Reflection

```
template <std::size_t I, typename T>
consteval decltype(auto) reference_by_index(T &val) noexcept {
    return val.[:
        nonstatic_data_members_of(
            ^^T,
            std::meta::access_context::current()
        ).at(I)
    :];
}
```

# Контракты



# Contracts

# Contracts

```
int isqrt(int n) noexcept;
```

# Contracts

```
int isqrt(int n) noexcept;

int main() {
    const auto x = isqrt(-4);
    return isqrt(x);
}
```

# Contracts

```
int isqrt(int n) noexcept;

int main() {
    const auto x = isqrt(-4);
    return isqrt(x);
}
```

# Contracts

```
int isqrt(int n) noexcept;

int main() {
    const auto x = isqrt(-4);
    return isqrt(x);
}
```

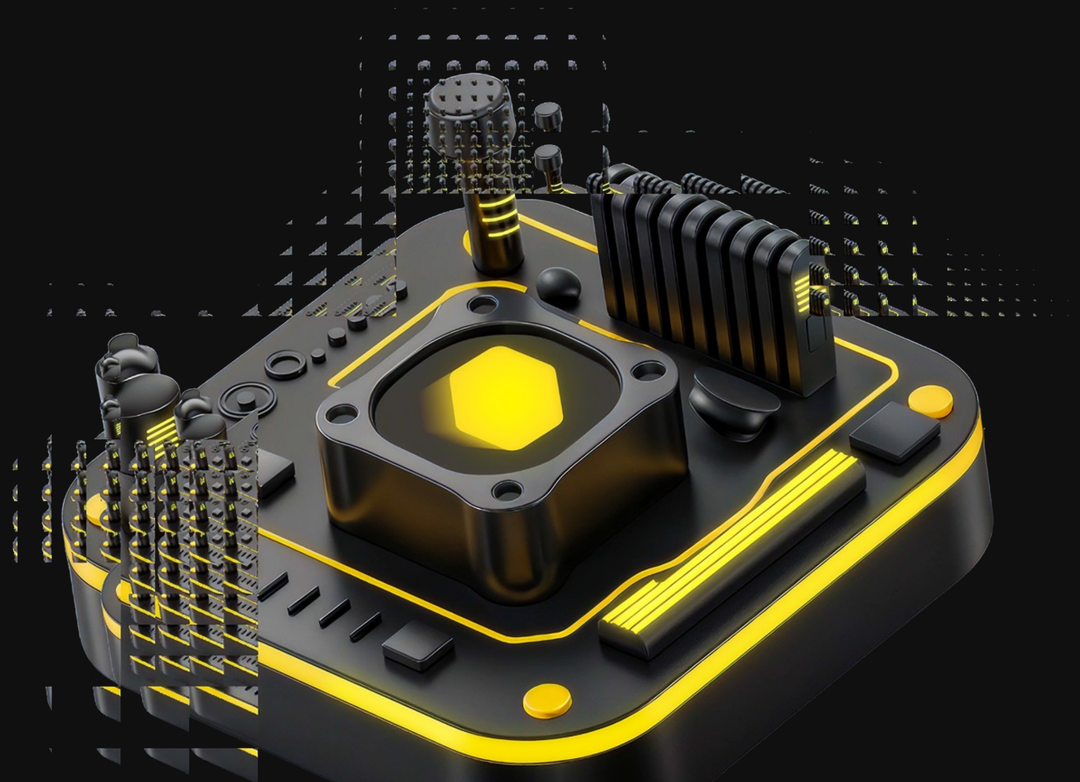
# Contracts

```
int isqrt(int n) noexcept pre(n >= 0);

int main() {
    const auto x = isqrt(-4);  // Warning/Error
    return isqrt(x);
}
```

# Contracts

```
int isqrt(int n) noexcept pre(n >= 0) post(r: r >= 0);

int main() {
    const auto x = isqrt(-4);  // Warning/Error
    return isqrt(x);  // Ok, no checks
}
```

# Ranges



# Ranges

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });
for (int n : result) {
    std::cout << n << ' '; // Output: -2 -4 -6 -8 -10
}
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });


return result;
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });


return result; // BUG
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });


return result; // BUG
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });

return result | std::ranges::to<std::vector>();
```

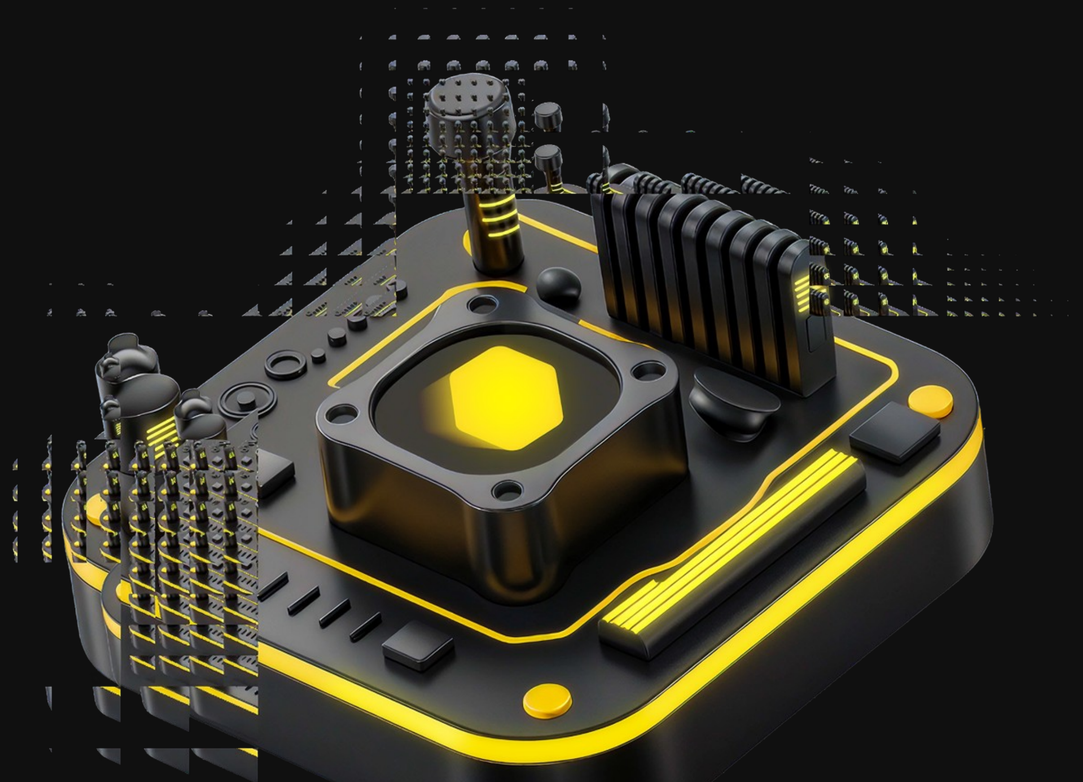
# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });

return result | std::ranges::to<std::vector>();
```

# Ranges

```
std::array<int, 10> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return -n; });

return result | std::ranges::to<std::vector>();
```

# Спасибо!

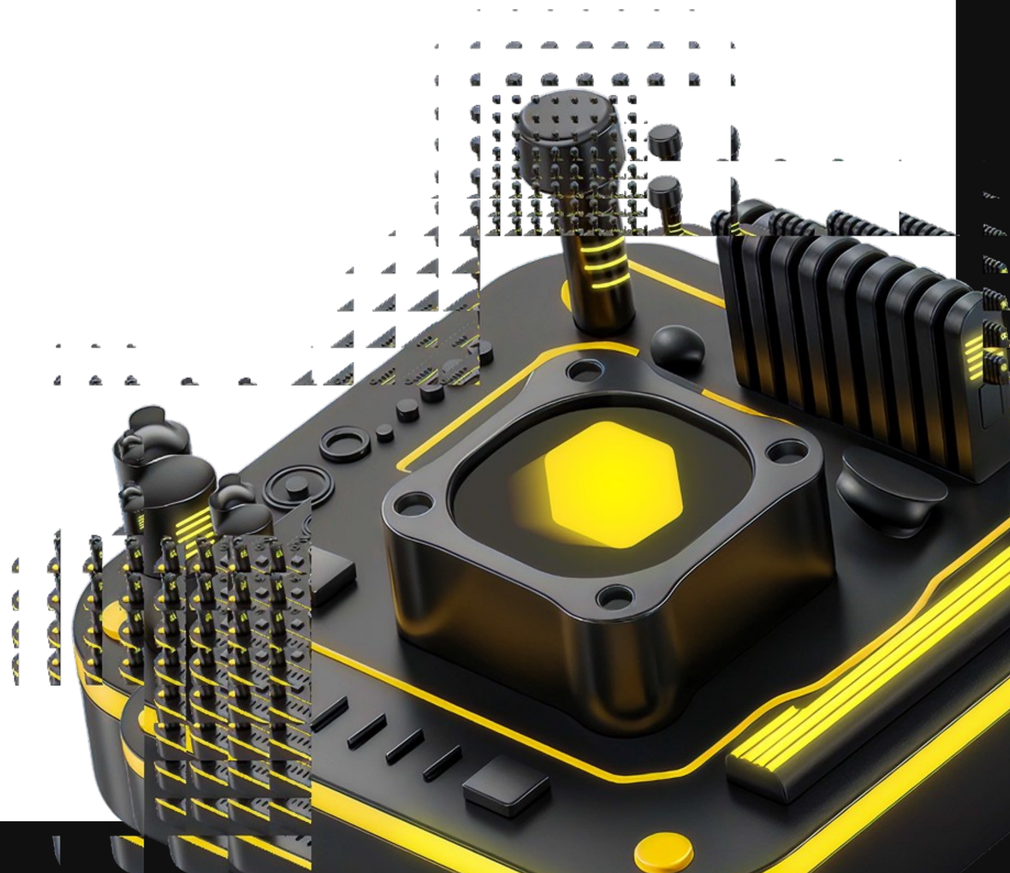


ГОРОДСКИЕ СЕРВИСЫ

ГОРОДСКИЕ СЕРВИСЫ

**название мероприятия**

# готов ответить на вопросы

# Антон Полухин

Эксперт разработчик C++